

Laboratorio 8

Teoría de la Computación

Adrian Lopez
Carné: 231361

Octubre, 2026

Este documento contiene las respuestas de los ejercicios sin código (parte a de los problemas 1, 2 y 3, y los problemas 4 y 5). La implementación, las tablas y las gráficas de la parte b están en el repositorio: <https://github.com/adirnnn/lab8-TCC>.

En todo el documento, $n/2$, $n/3$, etc. dentro del código en C son divisiones enteras, es decir $\lfloor n/2 \rfloor$, $\lfloor n/3 \rfloor$.

Problema 1 (parte a)

```
1 void function(int n) {  
2     int i, j, k, counter = 0;  
3     for (i = n/2; i <= n; i++) {  
4         for (j = 1; j+n/2 <= n; j++) {  
5             for (k = 1; k <= n; k = k*2) {  
6                 counter++;  
7             }  
8         }  
9     }  
10 }
```

Ciclo de i . Empieza en $\lfloor n/2 \rfloor$ y termina en n con incremento 1:

$$\text{vueltas}_i = n - \lfloor n/2 \rfloor + 1 \approx \frac{n}{2}.$$

Ciclo de j . La condición $j + \lfloor n/2 \rfloor \leq n$ es equivalente a $j \leq n - \lfloor n/2 \rfloor$. Como j empieza en 1:

$$\text{vueltas}_j = n - \lfloor n/2 \rfloor \approx \frac{n}{2}.$$

Este número no depende de i .

Ciclo de k . k toma los valores $1, 2, 4, \dots, 2^t$. Después de t vueltas $k = 2^t$, y el ciclo termina cuando $2^t > n$, es decir cuando $t > \log_2 n$:

$$\text{vueltas}_k = \lfloor \log_2 n \rfloor + 1.$$

Este número no depende ni de i ni de j .

Total. Como los tres ciclos son independientes, el número de veces que se ejecuta `counter++` es el producto:

$$T(n) = \sum_{i=\lfloor n/2 \rfloor}^n \sum_{j=1}^{n-\lfloor n/2 \rfloor} \sum_{t=0}^{\lfloor \log_2 n \rfloor} 1 = (n - \lfloor n/2 \rfloor + 1)(n - \lfloor n/2 \rfloor)(\lfloor \log_2 n \rfloor + 1)$$

$$\approx \frac{n}{2} \cdot \frac{n}{2} \cdot \log_2 n = \frac{n^2}{4} \log_2 n.$$

Por ejemplo, para $n = 10$: $(10 - 5 + 1)(10 - 5)(3 + 1) = 6 \cdot 5 \cdot 4 = 120$.

Para $n \geq 2$ se cumple $\frac{n^2}{4} \log_2 n \leq T(n) \leq (n + 1)^2 \log_2 n + (n + 1)^2 \leq 8n^2 \log_2 n$, por lo que $T(n) = \Theta(n^2 \log n)$. Descartando constantes:

$$T(n) \in O(n^2 \log n)$$

Problema 2 (parte a)

```

1 void function(int n) {
2     if (n <= 1) return;
3     int i, j;
4     for (i = 1; i <= n; i++) {
5         for (j = 1; j <= n; j++) {
6             printf("Sequence\n");
7             break;
8         }
9     }
10 }
```

Caso $n \leq 1$. La función hace una comparación y regresa: $\Theta(1)$.

Ciclo de i . Va de 1 a n : n vueltas.

Ciclo de j . Aunque su condición permitiría n vueltas, en la primera vuelta se ejecuta `printf` y luego `break`, que sale del ciclo interno. Por lo tanto, el ciclo de j hace una sola vuelta por cada i : se evalúa una vez $j \leq n$, se imprime una vez y se sale. Su costo es una constante c , no n .

Total. Sea c_0 el costo de la comparación inicial, c_1 el costo de cada evaluación de $i \leq n$ (se evalúa $n + 1$ veces) y c_2 el costo constante de una vuelta completa del ciclo externo (incremento de i , inicialización y una comparación de j , un `printf` y el `break`):

$$T(n) = c_0 + c_1(n + 1) + c_2 n = (c_1 + c_2)n + (c_0 + c_1).$$

El número de `printf` es exactamente n (para $n > 1$). Es una función lineal, así que:

$$T(n) \in O(n)$$

Nótese que, a pesar de tener dos ciclos anidados, **no** es $O(n^2)$ gracias al `break`.

Problema 3 (parte a)

```
1 void function(int n) {
2     int i, j;
3     for (i = 1; i <= n/3; i++) {
4         for (j = 1; j <= n; j += 4) {
5             printf("Sequence\n");
6         }
7     }
8 }
```

Ciclo de i . Va de 1 a $\lfloor n/3 \rfloor$:

$$\text{vueltas}_i = \lfloor n/3 \rfloor \approx \frac{n}{3}.$$

Ciclo de j . j toma los valores $1, 5, 9, \dots$, es decir $j = 1 + 4m$ con $m = 0, 1, 2, \dots$ mientras $1 + 4m \leq n$, o sea $m \leq (n-1)/4$:

$$\text{vueltas}_j = \left\lfloor \frac{n-1}{4} \right\rfloor + 1 \approx \frac{n}{4}.$$

No depende de i .

Total.

$$T(n) = \sum_{i=1}^{\lfloor n/3 \rfloor} \left(\left\lfloor \frac{n-1}{4} \right\rfloor + 1 \right) = \left\lfloor \frac{n}{3} \right\rfloor \left(\left\lfloor \frac{n-1}{4} \right\rfloor + 1 \right) \approx \frac{n}{3} \cdot \frac{n}{4} = \frac{n^2}{12}.$$

Por ejemplo, para $n = 10$: $3 \cdot (2 + 1) = 9$ impresiones. Las constantes $\frac{1}{3}$ y $\frac{1}{4}$ (y el paso de 4) no cambian el orden de crecimiento:

$$T(n) \in O(n^2)$$

Problema 4

En los tres algoritmos se cuenta el número de **comparaciones**, que es la operación dominante. El arreglo tiene n elementos.

Búsqueda lineal (Linear Search)

```
1 int linear_search(int A[], int n, int x) {
2     for (int i = 0; i < n; i++)
3         if (A[i] == x) return i;
4     return -1;
5 }
```

Mejor caso. x está en la primera posición: se hace 1 comparación.

$$T_{\text{mejor}}(n) = 1 \in \Theta(1).$$

Peor caso. x está en la última posición o no está en el arreglo: se comparan los n elementos.

$$T_{\text{peor}}(n) = n \in \Theta(n).$$

Caso promedio. Suponemos que x está en el arreglo y que es igual de probable que esté en cualquier posición i (probabilidad $\frac{1}{n}$). Si está en la posición i (contando desde 1) se hacen i comparaciones:

$$T_{\text{prom}}(n) = \sum_{i=1}^n i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} \in \Theta(n).$$

Si además x no está con probabilidad q , entonces $T_{\text{prom}}(n) = (1-q)\frac{n+1}{2} + qn$, que sigue siendo $\Theta(n)$.

Búsqueda binaria (Binary Search)

Requiere que el arreglo esté ordenado. En cada paso se compara x con el elemento del medio y se descarta la mitad donde no puede estar.

Mejor caso. x es el elemento del medio en la primera comparación.

$$T_{\text{mejor}}(n) = 1 \in \Theta(1).$$

Peor caso. x no está o se encuentra en el último paso. Cada comparación reduce el tamaño a la mitad:

$$T(n) = T\left(\frac{n}{2}\right) + 1, \quad T(1) = 1.$$

Expandiendo:

$$T(n) = T\left(\frac{n}{2}\right) + 1 = T\left(\frac{n}{4}\right) + 2 = \cdots = T\left(\frac{n}{2^k}\right) + k.$$

Se llega al caso base cuando $\frac{n}{2^k} = 1$, es decir $k = \log_2 n$:

$$T_{\text{peor}}(n) = \lfloor \log_2 n \rfloor + 1 \in \Theta(\log n).$$

Caso promedio. Tomamos $n = 2^k - 1$ (árbol de decisión completo de k niveles) y suponemos que x está en el arreglo con la misma probabilidad en cada posición. En el nivel i del árbol hay 2^{i-1} elementos y cada uno se encuentra con exactamente i comparaciones. Entonces:

$$T_{\text{prom}}(n) = \frac{1}{n} \sum_{i=1}^k i \cdot 2^{i-1}.$$

Usando la identidad $\sum_{i=1}^k i 2^{i-1} = (k-1)2^k + 1$ (se verifica por inducción: para $k=1$ da 1, y al pasar de k a $k+1$ se suma $(k+1)2^k$, obteniendo $(k-1)2^k + 1 + (k+1)2^k = k2^{k+1} + 1$) y que $2^k = n+1$:

$$T_{\text{prom}}(n) = \frac{(k-1)(n+1) + 1}{n} \approx k-1 = \log_2(n+1) - 1 \in \Theta(\log n).$$

Es decir, en promedio se hace aproximadamente una comparación menos que en el peor caso.

Quick sort

Se elige un pivote y se particiona el arreglo en los elementos menores y mayores que el pivote, lo que cuesta $n - 1$ comparaciones. Luego se ordena recursivamente cada parte. Si el pivote queda en la posición k (con k elementos a su izquierda):

$$T(n) = T(k) + T(n - 1 - k) + (n - 1), \quad T(0) = T(1) = 0.$$

Mejor caso. El pivote siempre queda en la mitad (es la mediana), así que las dos partes tienen tamaño $\approx n/2$:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn.$$

Por el teorema maestro con $a = 2$, $b = 2$, $f(n) = cn = \Theta(n^{\log_2 2})$ (caso 2): $T(n) = \Theta(n \log n)$. Intuitivamente: el árbol de recursión tiene $\log_2 n$ niveles y en cada nivel se hacen cn comparaciones en total.

$$T_{\text{mejor}}(n) \in \Theta(n \log n).$$

Peor caso. El pivote siempre es el mínimo o el máximo (por ejemplo, si el arreglo ya está ordenado y se toma como pivote el primer o el último elemento). Una parte queda vacía y la otra con $n - 1$ elementos:

$$T(n) = T(n - 1) + (n - 1) = \sum_{i=1}^{n-1} i = \frac{n(n - 1)}{2}.$$

$$T_{\text{peor}}(n) \in \Theta(n^2).$$

Caso promedio. Suponemos que el pivote tiene la misma probabilidad $\frac{1}{n}$ de quedar en cualquier posición $k = 0, \dots, n - 1$:

$$T(n) = (n - 1) + \frac{1}{n} \sum_{k=0}^{n-1} [T(k) + T(n - 1 - k)] = (n - 1) + \frac{2}{n} \sum_{k=0}^{n-1} T(k).$$

Multiplicando por n :

$$nT(n) = n(n - 1) + 2 \sum_{k=0}^{n-1} T(k). \quad (1)$$

La misma ecuación para $n - 1$:

$$(n - 1)T(n - 1) = (n - 1)(n - 2) + 2 \sum_{k=0}^{n-2} T(k). \quad (2)$$

Restando (1) - (2):

$$nT(n) - (n - 1)T(n - 1) = 2(n - 1) + 2T(n - 1) \implies nT(n) = (n + 1)T(n - 1) + 2(n - 1).$$

Dividiendo entre $n(n + 1)$:

$$\frac{T(n)}{n + 1} = \frac{T(n - 1)}{n} + \frac{2(n - 1)}{n(n + 1)} = \frac{T(n - 1)}{n} + \frac{4}{n + 1} - \frac{2}{n}.$$

Sea $S(n) = \frac{T(n)}{n+1}$ con $S(1) = 0$. Sumando de forma telescópica (el término con $i = 1$ vale $\frac{4}{2} - \frac{2}{1} = 0$) y usando $H_n = \sum_{i=1}^n \frac{1}{i}$:

$$S(n) = \sum_{i=1}^n \left(\frac{4}{i+1} - \frac{2}{i} \right) = 4(H_{n+1} - 1) - 2H_n = 2H_n + \frac{4}{n+1} - 4.$$

Por lo tanto:

$$T(n) = (n+1)S(n) = 2(n+1)H_n - 4n.$$

Verificación: $T(2) = 2 \cdot 3 \cdot \frac{3}{2} - 8 = 1$ y $T(3) = 2 \cdot 4 \cdot \frac{11}{6} - 12 = \frac{8}{3}$, que coinciden con la recurrencia. Como $H_n \approx \ln n$:

$$T_{\text{prom}}(n) \approx 2n \ln n \approx 1.39 n \log_2 n \in \Theta(n \log n).$$

Resumen

Algoritmo	Mejor caso	Caso promedio	Peor caso
Búsqueda lineal	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Búsqueda binaria	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Quick sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$

Problema 5

Recordemos las definiciones:

- $f(n) = O(g(n))$ si existen $c > 0$ y n_0 tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.
- $f(n) = \Omega(g(n))$ si existen $c > 0$ y n_0 tales que $f(n) \geq c g(n)$ para todo $n \geq n_0$.
- $f(n) = \Theta(g(n))$ si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$.

a) Si $f(n) = \Theta(g(n))$ y $g(n) = \Theta(h(n))$, entonces $h(n) = \Theta(f(n))$.

Verdadero.

Por hipótesis existen constantes positivas tales que:

$$\begin{aligned} c_1 g(n) &\leq f(n) \leq c_2 g(n) && \text{para } n \geq n_1, \\ c_3 h(n) &\leq g(n) \leq c_4 h(n) && \text{para } n \geq n_2. \end{aligned}$$

Sustituyendo la segunda en la primera, para $n \geq \max(n_1, n_2)$:

$$c_1 c_3 h(n) \leq f(n) \leq c_2 c_4 h(n).$$

Esto ya dice que $f(n) = \Theta(h(n))$ (transitividad). Despejando $h(n)$:

$$\frac{1}{c_2 c_4} f(n) \leq h(n) \leq \frac{1}{c_1 c_3} f(n),$$

y como $\frac{1}{c_2 c_4} > 0$ y $\frac{1}{c_1 c_3} > 0$, se cumple $h(n) = \Theta(f(n))$ (simetría). En resumen, Θ es transitiva y simétrica.

b) Si $f(n) = O(g(n))$ y $g(n) = O(h(n))$, entonces $h(n) = \Omega(f(n))$.

Verdadero.

Por hipótesis:

$$\begin{aligned} f(n) &\leq c_1 g(n) \quad \text{para } n \geq n_1, \\ g(n) &\leq c_2 h(n) \quad \text{para } n \geq n_2. \end{aligned}$$

Entonces, para $n \geq \max(n_1, n_2)$:

$$f(n) \leq c_1 c_2 h(n),$$

o sea $f(n) = O(h(n))$ (transitividad de O). Despejando:

$$h(n) \geq \frac{1}{c_1 c_2} f(n) \quad \text{para } n \geq \max(n_1, n_2),$$

con $\frac{1}{c_1 c_2} > 0$, que es exactamente la definición de $h(n) = \Omega(f(n))$. (En general, $f = O(h) \iff h = \Omega(f)$.)

c) $f(n) = \Theta(n^2)$, donde $f(n)$ es el tiempo de ejecución de $A(n)$.

```

1 def A(n):
2     atupla = tuple(range(0, n))
3     S = set()
4     for i in range(0, n):
5         for j in range(i + 1, n):
6             S.add(atupla[i:j])

```

Falso. En realidad $f(n) = \Theta(n^3)$.

Los dos ciclos recorren $\frac{n(n-1)}{2}$ pares (i, j) , lo que daría $\Theta(n^2)$ *solo si* el trabajo de cada par fuera constante. Pero no lo es:

- `atupla[i:j]` crea una tupla **nueva** copiando los $j - i$ elementos: cuesta $\Theta(j - i)$.
- `S.add(...)` tiene que calcular el hash de la tupla, y el hash de una tupla se calcula recorriendo sus $j - i$ elementos: cuesta $\Theta(j - i)$. (Además, todas las tuplas son distintas, así que se guardan todas en el set.)

Las líneas `tuple(range(0, n))` y `set()` cuestan $O(n)$, que es despreciable. Entonces el costo total es proporcional a:

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (j - i) = \sum_{i=0}^{n-1} \sum_{m=1}^{n-1-i} m = \sum_{L=0}^{n-1} \frac{L(L+1)}{2} = \frac{(n-1)n(n+1)}{6} = \frac{n^3 - n}{6},$$

donde $L = n - 1 - i$. Por ejemplo, para $n = 3$ los pares son $(0, 1), (0, 2), (1, 2)$ con longitudes $1 + 2 + 1 = 4 = \frac{27-3}{6}$.

Por lo tanto $f(n) = \Theta(n^3)$. Como $n^3 \notin O(n^2)$ (no existe c tal que $n^3 \leq c n^2$ para todo n grande, porque $n^3/n^2 = n \rightarrow \infty$), se concluye que $f(n) \neq \Theta(n^2)$. (Sí es cierto que $f(n) = \Omega(n^2)$, pero no $O(n^2)$.)

Comprobación experimental. Se midió $A(n)$ en Python 3.11:

n	tiempo (s)	t/n^2	t/n^3
100	0.0027	2.68×10^{-7}	2.68×10^{-9}
200	0.0153	3.83×10^{-7}	1.92×10^{-9}
400	0.1149	7.18×10^{-7}	1.80×10^{-9}
800	0.7974	1.25×10^{-6}	1.56×10^{-9}

Al duplicar n , el tiempo se multiplica aproximadamente por 7-8 ($\approx 2^3$). El cociente t/n^2 sigue creciendo, mientras que t/n^3 se mantiene casi constante, lo cual confirma el crecimiento cúbico.